Дисциплина: метрология программного обеспечения

ЛАБОРАТОРНАЯ РАБОТА № 2

Метрики сложности потока управления

Анализ программы на TypeScript по метрике Джилба

Выполнили: Пузик Владислав

Хренков Даниил

Группа: 451001

2026

1. Цель и теоретические сведения

Цель: подготовить программу со всеми видами циклов TypeScript и ветвлениями, построить подробную схему, вручную рассчитать метрику Джилба и проверить результат программным анализатором.

Метрика Джилба

Абсолютная сложность CL — количество ветвлений исходной структуры. Учитываются if, каждый цикл и каждый case. Относительная сложность cl = CL / N, где N — число операторов по принятому профилю. Максимальная вложенность CLI — наибольший уровень ветвления; внешний уровень равен нулю. При CL > 0 выполняется CL > CLI; без ветвлений оба равны нулю.

Для расчёта вложенности switch представляется цепочкой вложенных if. Селектор и default не добавляют условий. Для k case глубины сравнений равны d, d+1, …, d+k−1. Тело j-го case имеет глубину d+j (j с 1). Поэтому отдельный switch с четырьмя case даёт CL=4 и CLI=3. Else и завершающий while в do-while не считаются отдельными ветвлениями.

В N входят самостоятельные выражения, объявления переменных, управляющие операторы и переходы. Блоки, метки, типы и объявления функций не учитываются; тела функций анализируются. Заголовок for входит в один оператор цикла. При N=0 относительная сложность не определена.

Метрика Маккейба

Цикломатическая сложность характеризует число линейно независимых путей графа потока управления: V(G)=E−V+2 для одной функции с нормализованными входом и выходом; для P таких компонент — E−V+2P. E — число дуг, V — число вершин. При бинарных решениях V(G)=D+1. Многовыходный узел добавляет число исходов минус один.

В связном планарном изображении сложность также равна числу областей, включая внешнюю. Это размер базиса путей, а не число всех возможных трасс. Циклы допускают неограниченное число трасс; некоторые пути могут быть невыполнимы из-за зависимых условий. Метрика помогает планировать структурное тестирование, но не гарантирует качество программы.

Для схемы данного примера D=12, поэтому V(G)=13. Это теоретический расчёт: приложение реализует Джилба. При раскрытии короткого замыкания логических выражений граф и D могут отличаться. Одинаковая V(G) не означает одинаковую вложенность. Джилб и Маккейб не измеряют полноту требований, сложность данных или предметной области.

2. Исходный код анализируемой программы

Файл example.ts. Программа использует пять видов циклов: for, while, do-while, for-of и for-in. Числа слева — номера строк, они не входят в код.

```
01  let total = 0;
02  const values: number[] = [1, 2, 3];
03  const labels = { a: "A", bb: "B" };
04  for (let i = 0; i < 4; ++i) {
05      if (i > 0) {
06          switch (i) {
07              case 1:
08                  if (total === 0) {
09                      total += 2;
10                  } else {
11                      total += 1;
12                  }
13                  break;
14              case 2:
15                  total += 3;
16                  break;
17              case 3:
18                  total += 4;
19                  break;
20              case 4:
21                  total += 5;
22                  break;
23              default:
24                  total += 6;
25          }
26      } else {
27          total += 1;
28      }
29  }
30  let j = 0;
31  while (j < 2) {
32      total += j;
33      ++j;
34  }
35  let k = 0;
36  do {
37      total += k;
38      ++k;
39  } while (k < 2);
40  for (const value of values) {
41      total += value;
42  }
43  for (const key in labels) {
44      total += key.length;
45  }
46  if (total < 0) {
47      total = 0;
48  }
49  console.log(`total = ${total}`);
```

При отдельном запуске выводится total = 20. Метрики статические: количество исполненных итераций и достижимость отдельных case при выбранных константах не меняют структуру исходного текста.

3. Ручной расчёт

Строка	Ветвление	Уровень
4	for	0
5	if (i > 0)	1
7	case 1	2
8	if (total === 0)	3
14	case 2	3
17	case 3	4
20	case 4	5
31	while	0
36	do-while	0
40	for-of	0
43	for-in	0
46	if (total < 0)	0

CL = 5 циклов + 3 if + 4 case = 12. Максимум в таблице: CLI = 5. Вложенный if первого case имеет уровень 3; добавлять его глубину ко всему switch нельзя.

Категория	Строки исходника	Количество
Объявления	1, 2, 3, 30, 35	5
Выражения	9, 11, 15, 18, 21, 24, 27, 32, 33, 37, 38, 41, 44, 47, 49	15
if	5, 8, 46	3
Циклы	4, 31, 36, 40, 43	5
switch	6	1
break	13, 16, 19, 22	4

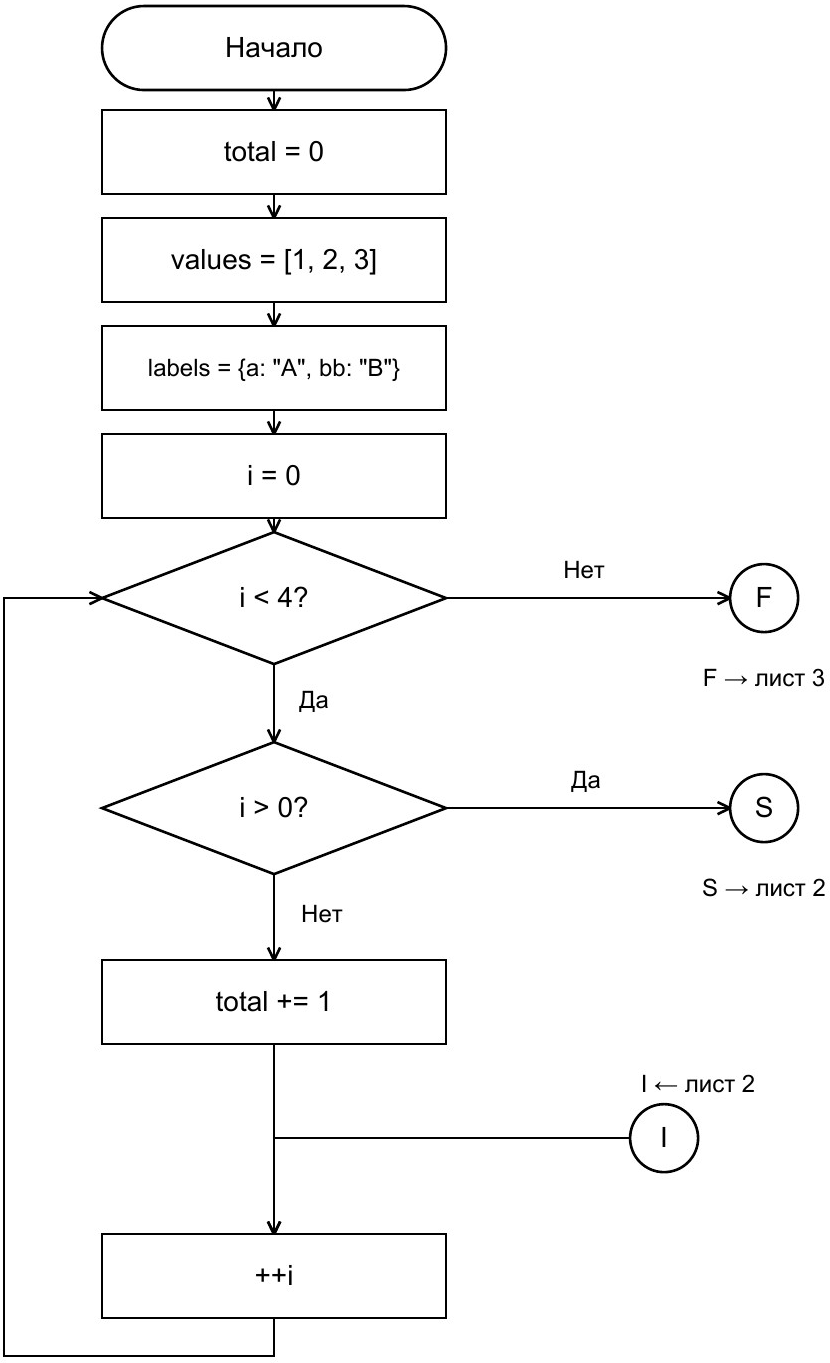
N = 5 + 15 + 3 + 5 + 1 + 4 = 33.

cl = 12 / 33 = 4 / 11 ≈ 0,363636; CL = 12; CLI = 5.

Проверка: 12 > 5. Инициализация let i=0 внутри for не является самостоятельным Statement в TS. Скрытые r, t и keys на раскрытой схеме в N не входят. Схема следует на четырёх листах; одинаковые буквы соединителей обозначают продолжение потока.

Инициализация и внешний цикл for

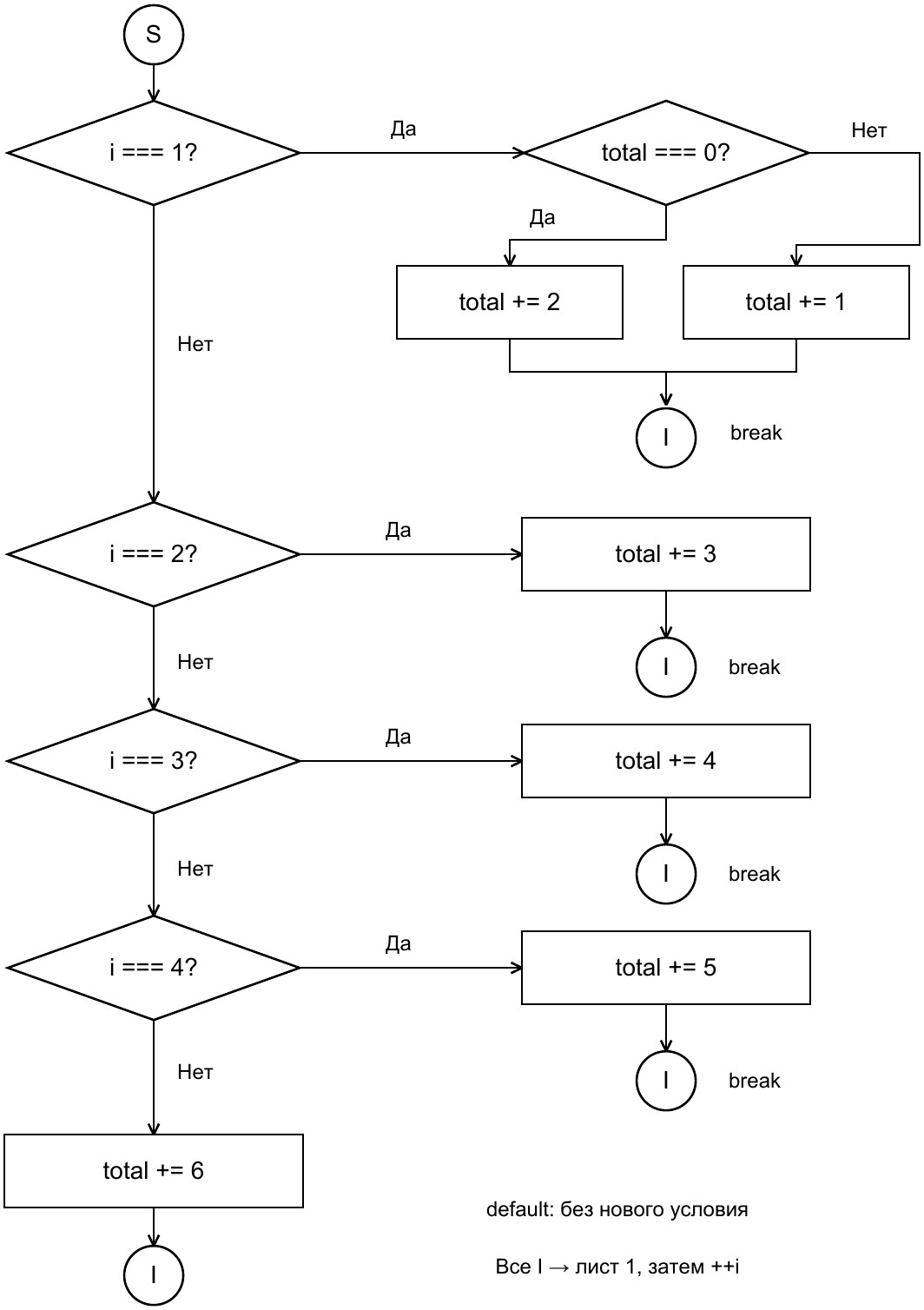
Схема example.ts · лист 1 из 4



Соединители с одной буквой обозначают продолжение потока.

switch как цепочка вложенных if

Схема example.ts · лист 2 из 4

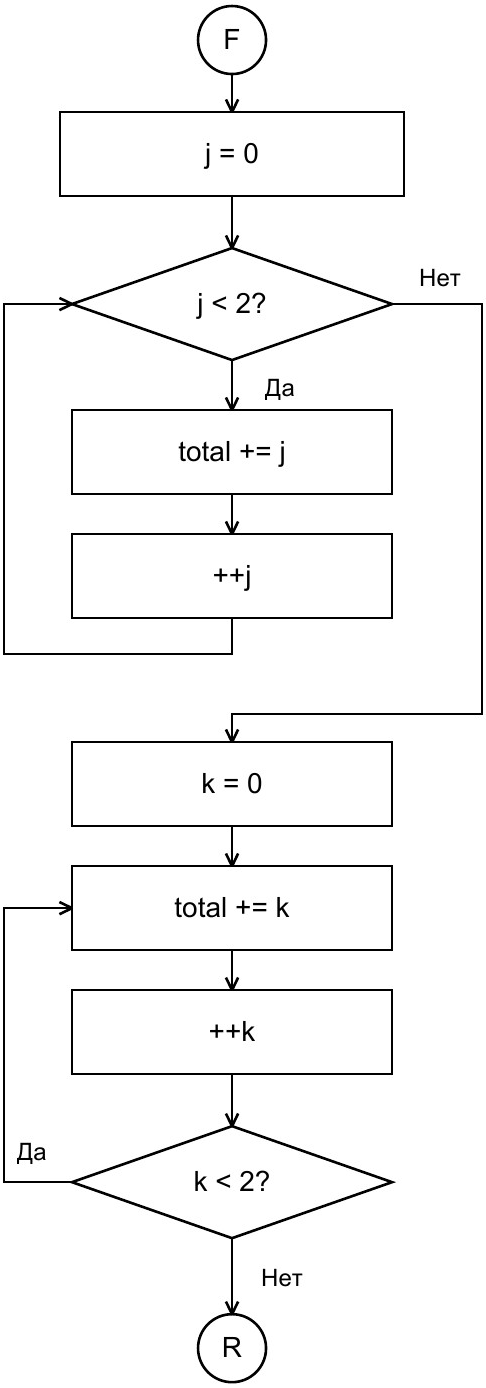


default: без нового условия

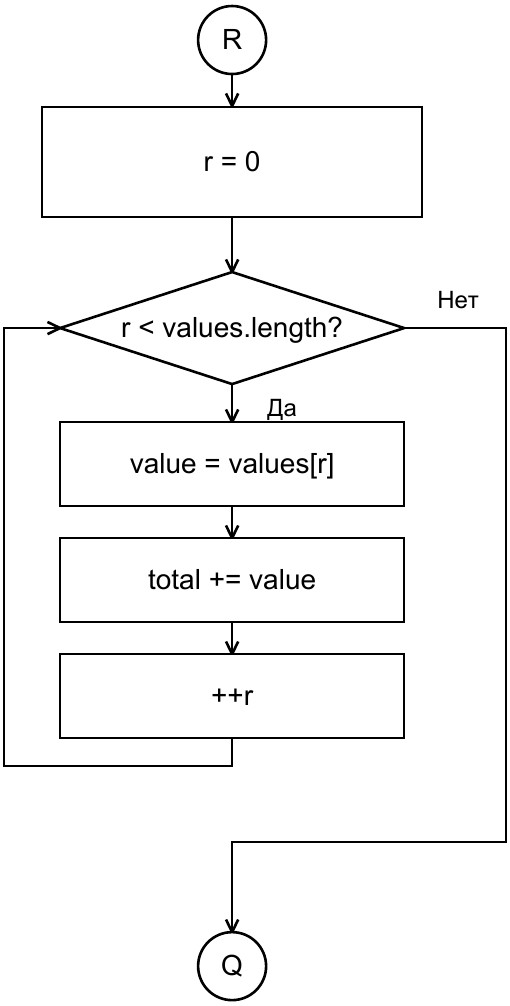
Все I → лист 1, затем ++i

Циклы while, do-while и for-of

Схема example.ts · лист 3 из 4



R: переход в правый столбец.

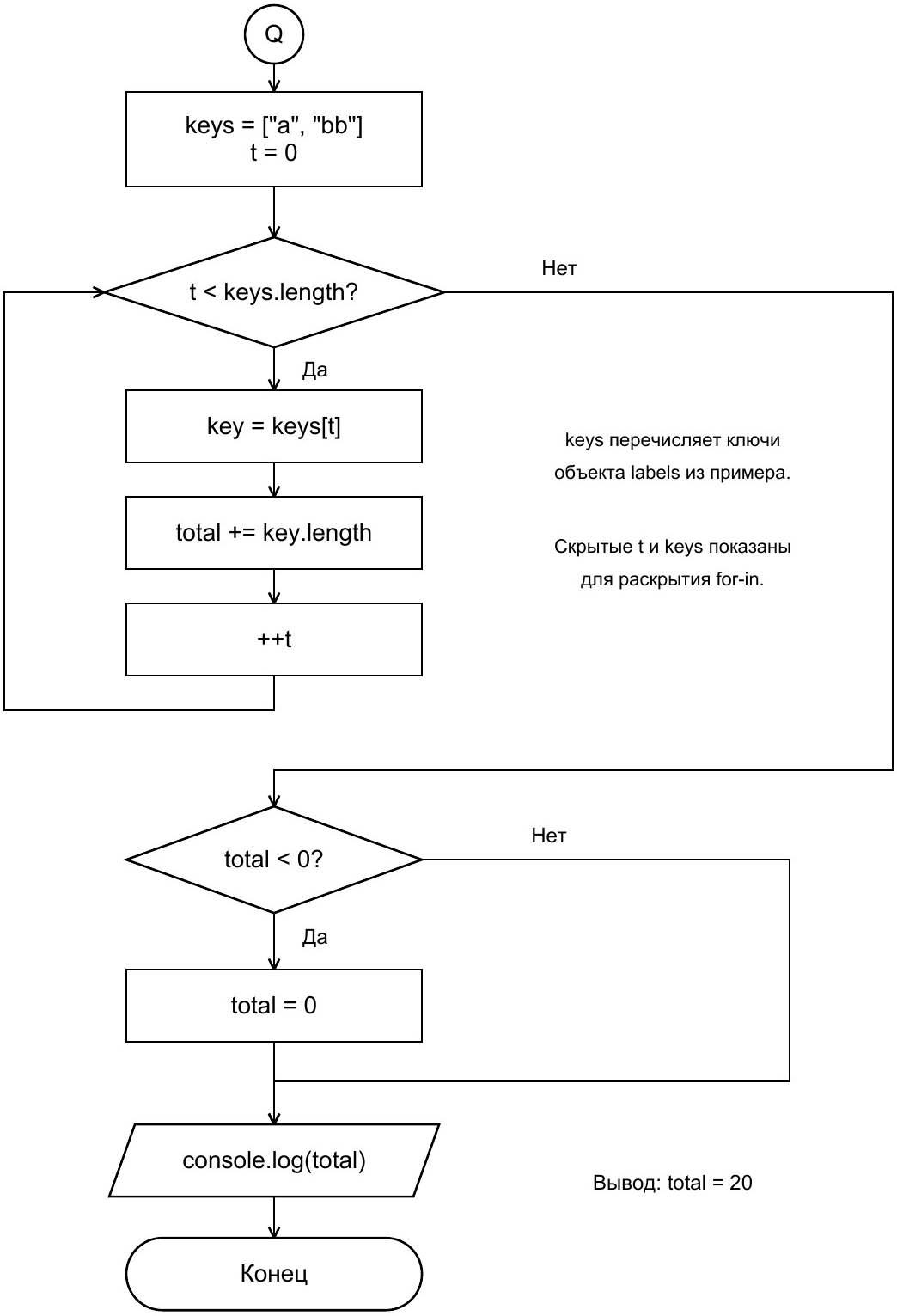


Q → лист 4

r раскрывает for-of массива.

Цикл for-in и завершение

Схема example.ts · лист 4 из 4



4. Программный расчёт и проверка

Графический анализатор разбирает example.ts через TypeScript Compiler API 5.9.3 в браузере. Синтаксическое дерево обходится рекурсивно; у каждого case отдельно вычисляется глубина тела. Исходный код не исполняется, импорты не загружаются.

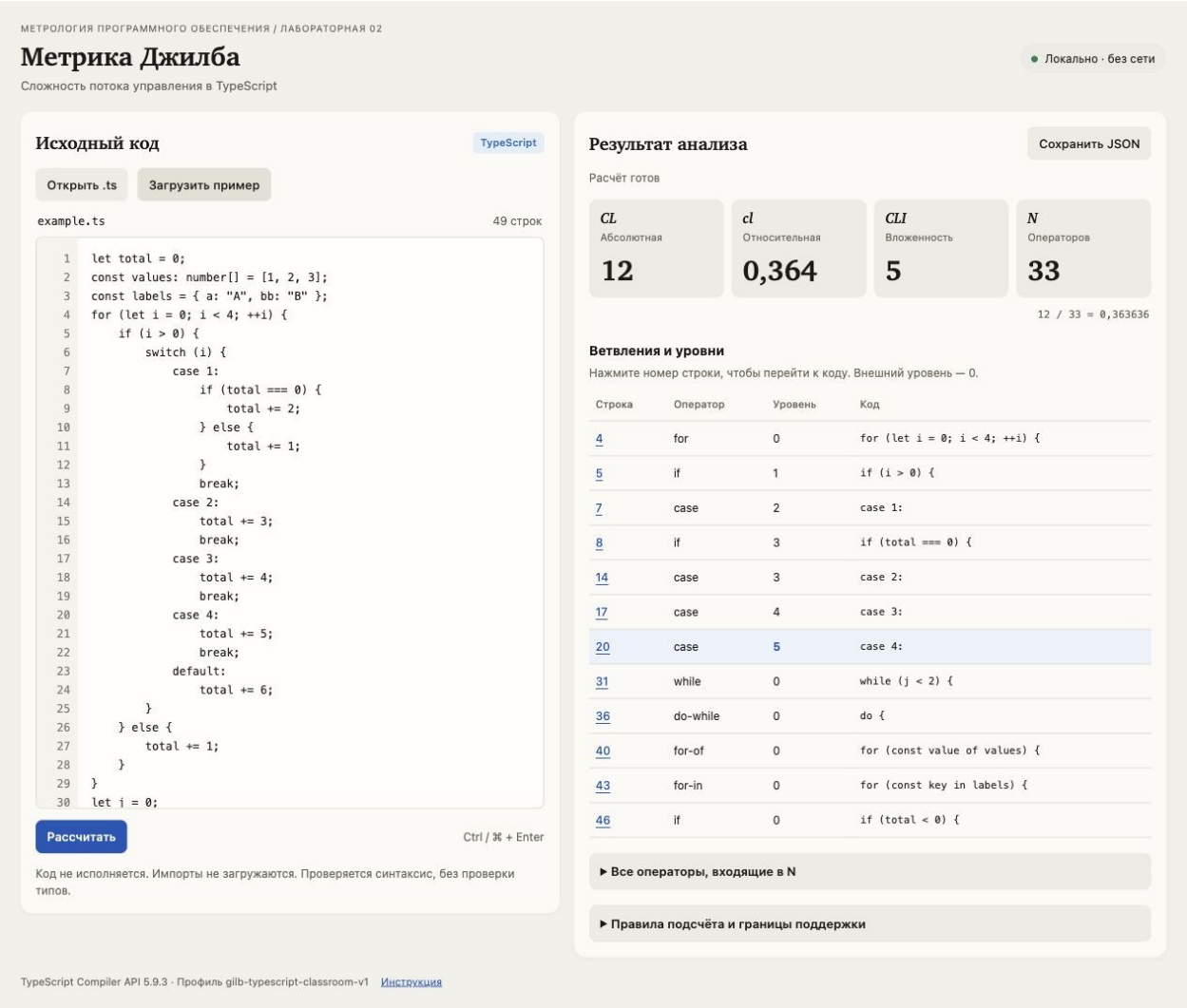


Рисунок — фактический результат работы анализатора на example.ts.

Показатель	Вручную	Программой
CL	12	12
N	33	33
cl	12/33 ≈ 0,363636	0,363636
CLI	5	5

Проверены: if в первом и последнем case, вложенный switch, default в середине, fallthrough, else без фигурных скобок, все циклы, ASI, комментарии, строки, регулярные выражения, шаблонные строки и независимая вложенность функций. Синтаксические ошибки и неподдерживаемые конструкции показываются вместо результатов.

5. Вывод

Ограничения профиля: не поддерживаются классы, методы объектов, try/catch/finally, генераторы, метки управления, namespace/enum и tagged templates. Проверяется синтаксис, но не типы/существование имён. Приложение не является универсальным анализатором всех расширений TypeScript.

Созданы анализируемая программа на TypeScript, графический анализатор метрики Джилба и подробная схема. Ручной и программный расчёты совпали: CL=12, N=33, cl=12/33, CLI=5. Самая глубокая точка — сравнение четвёртого case. Вложенный if первого case не увеличивает этот максимум.

Контрольные вопросы

Почему do-while даёт одно ветвление? Потому что это один оператор цикла с проверкой после тела.

Что изменит перенос if из первого case в четвёртый? CL и N останутся прежними, CLI увеличится с 5 до 6.

Почему default не добавляет CL? В нём не проверяется новое условие; собственные if/циклы в его теле учитываются.

Почему N не равен числу точек с запятой? Заголовки циклов, ASI и составные операторы определяются грамматикой.

Чем for-in отличается от for-of? Первый перечисляет строковые ключи, второй получает значения итератора.

Почему Маккейб не равен числу всех путей? Он задаёт размер независимого базиса, а циклы допускают повторение трасс.